

MODULE 14 · DOMAIN 2 - DESIGN FOR NEW SOLUTIONS

Choosing Compute

EC2 appears in half the question bank. Rarely as the topic.

DOMAIN 2 of 4 EXAM WEIGHT 29% MODULE 14 of 44

At professional level you rarely choose an instance type. You choose a **shape**, and let AWS pick the types.

Instance families, by what they are for

FAMILY	OPTIMISED FOR	TYPICAL USE
T	Burstable, cheap baseline	Small or idle-heavy workloads
M	Balanced	The default when nothing dominates
C	Compute	Batch, encoding, HPC, gaming servers
R and X	Memory	In-memory caches, large databases
I and D	Storage throughput and IOPS	NoSQL, data warehouses, local scratch
P and G	Accelerators	Training, inference, graphics

ON THE EXAM

T instances accrue credits and can throttle when exhausted. Sustained load on a T instance is a classic misdiagnosed performance problem.

Graviton

- ◆ **AWS-designed arm64 processors**, consistently better price-performance than equivalent x86 families.
- ◆ Generations run from Graviton2 to **Graviton5**, which reached general availability in mid-2026.
- ◆ **arm64 and Linux only**. No Windows, and no GPU-attached instance types.
- ◆ Managed services run on it too, including RDS, Aurora, ElastiCache, OpenSearch and Lambda, with no porting work.

ON THE EXAM

Interpreted and JIT languages port with a rebuild. Compiled binaries and native dependencies are where migrations actually stall.

Describing a fleet rather than listing it

- ◆ **Attribute-based instance selection** expresses requirements as vCPUs, memory and storage, and AWS picks matching types.
- ◆ New instance families qualify automatically as AWS releases them. A hard-coded list never does.
- ◆ **Mixed instances policies** combine On-Demand and Spot in one group, with a base count and a percentage above it.
- ◆ Both together let one Auto Scaling group span x86 and arm64, which massively widens the capacity you can draw on.

ON THE EXAM

Diversity is availability. The more instance types and Availability Zones a group can use, the less a shortage in any one pool matters.

The scaling policy types

POLICY	REACTS TO	CHOOSE IT WHEN
Target tracking	A metric against a target	The default. Keep CPU near 60%
Step scaling	Alarm breach size	Different responses to different severities
Simple scaling	One alarm, with a cooldown	Legacy. Prefer the two above
Scheduled	The clock	Known events: market open, a sale
Predictive	Forecast from history	Recurring daily or weekly patterns

ON THE EXAM

Reactive policies scale after demand arrives. Only scheduled and predictive scaling can have capacity ready beforehand.

Reactive or ahead of demand?

TARGET TRACKING

Responds to what happened

- Simple to configure and reason about
- Always lags the demand curve
- Lag equals detection plus boot time
- Fine for gradual change

PREDICTIVE SCALING

Acts on what usually happens

- Forecasts from at least 24h of history
- Provisions before the rise
- Needs a repeating pattern to work
- Runs alongside target tracking

THE TIEBREAKER

Use both. Predictive handles the pattern it has seen; target tracking catches the day the pattern is wrong.

Making instances ready sooner

- ◆ **Warm pools** keep pre-initialised instances stopped or hibernated, so scale-out skips boot and application startup.
- ◆ Stopped instances cost only storage. This is the answer when boot time, not scaling logic, is the problem.
- ◆ **Lifecycle hooks** pause an instance entering or leaving service, so you can bootstrap or drain.
- ◆ A **golden AMI** built with EC2 Image Builder removes most startup work in the first place.

ON THE EXAM

If the complaint is that scaling reacts too slowly, ask what is slow. A long boot needs a warm pool; a late trigger needs a different policy.

Placement groups

- ◆ **Cluster** packs instances close together in one AZ for lowest latency and highest throughput between them.
- ◆ Cluster concentrates risk. Everything is in one AZ, and a capacity shortfall can fail the whole launch.
- ◆ **Spread** places each instance on distinct hardware. Limited to seven per AZ, for small critical sets.
- ◆ **Partition** groups instances into partitions on separate racks. For HDFS, Cassandra, Kafka.

ON THE EXAM

Cluster optimises for speed between nodes. Spread and partition optimise for not losing them together. Read which the scenario wants.

Spot without pain

- ◆ **Use price-capacity-optimized.** AWS now advises against lowest-price, because replacements land in pools that get interrupted again.
- ◆ **Diversify hard.** Many instance types across many AZs is what actually reduces interruptions.
- ◆ **Capacity Rebalancing** acts on a rebalance recommendation, often arriving before the two-minute notice.
- ◆ Handle the interruption notice: drain, checkpoint, and hand work back to a queue.

ON THE EXAM

Spot Fleet is now legacy. New designs use an Auto Scaling group or EC2 Fleet, which get new capabilities first.

Reading the scenario

THE SCENARIO SAYS

Scaling is too slow, boot takes minutes

Traffic spikes at the same time daily

Drain connections before termination

Nodes need low latency between them

Frequent Spot interruptions

Cut compute cost, Linux, no GPU

IT IS POINTING AT

Warm pool, or a golden AMI

Predictive or scheduled scaling

A lifecycle hook

Cluster placement group

price-capacity-optimized, plus diversity

Graviton

ON THE EXAM

Separate the trigger from the delay. Many scaling complaints are about how long an instance takes to be useful, not about the policy.

Meridian Freight's tracking service scales on CPU with target tracking at 60%. During sudden load, requests time out for roughly seven minutes before new capacity helps. Instances take about six minutes to boot, install dependencies and warm a local cache.

The scaling alarm itself fires within a minute.

What should the solutions architect recommend?

- ☐ A Lower the target tracking threshold from 60% to 40%
- ☐ B Add a warm pool of pre-initialised stopped instances, and bake dependencies into a golden AMI
- ☐ C Switch from target tracking to step scaling
- ☐ D Increase the Auto Scaling group's maximum size

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A

 Lower the target
Buys headroom by running more instances permanently, and every scale-out still takes six minutes. It pays continuously to hide a delay.
- B

 Warm pool plus golden AMI
Attacks the six minutes directly. A stopped, pre-initialised instance joins in seconds instead.
- C

 Step scaling
Changes how much capacity is added, not how fast it becomes useful. The six-minute boot is unchanged.
- D

 Raise the maximum
The group was never at its ceiling. Nothing in the scenario suggests it was capped.

THE SIGNAL

One minute to trigger, six minutes to boot. The scenario hands you the arithmetic so you attack the right number.

A retail platform sees traffic rise sharply at 09:00 every weekday and fall after 18:00, with a much larger peak on Mondays. Target tracking scales correctly but always about ten minutes late, and the first ten minutes of each morning are degraded.

The pattern has been stable for two years.

What should the solutions architect recommend?

- ☐ A Enable predictive scaling alongside the existing target tracking policy
- ☐ B Replace target tracking with a scheduled action at 08:45 every weekday
- ☐ C Replace target tracking with step scaling on a lower threshold
- ☐ D Run the peak instance count all day

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A

Predictive plus target tracking
Predictive provisions ahead of a pattern it has learned, including the larger Monday peak, and target tracking still covers unexpected days.
- B

Scheduled action only
The strong near miss. It handles 09:00, and one fixed number cannot express a bigger Monday, and replacing target tracking removes the safety net.
- C

Step scaling
Still reactive. It changes the size of the response, not whether capacity exists before demand arrives.
- D

Peak all day
Solves it with money, paying for the evening and weekend peak that never comes.

THE SIGNAL

The Monday detail is planted. A scheduled action is one number; a forecast varies by day, which is exactly what the pattern needs.

A batch rendering fleet runs on Spot across two instance types in one Availability Zone, using the lowest-price allocation strategy. Interruptions are frequent, and replacements are often interrupted within minutes of launching.

Jobs take about 40 minutes and currently restart from the beginning.

What should the solutions architect change?

- ☐ A Move the fleet to On-Demand instances
- ☐ B Use price-capacity-optimized, diversify across many types and AZs, enable Capacity Rebalancing, and checkpoint work
- ☐ C Set a higher maximum Spot price so instances are not outbid
- ☐ D Use a cluster placement group to improve capacity availability

◆ PAUSE AND DECIDE

What should the solutions architect change?

A Move to On-Demand

Removes interruptions and most of the saving. Rendering is the archetypal Spot workload once it checkpoints.

C Higher maximum price

Spot pricing has not worked by bidding for years. Interruptions come from capacity demand, not from being outbid.

B Strategy, diversity, rebalancing, checkpoints

Fixes the cause and the symptom. Deeper pools mean fewer interruptions, and checkpoints mean an interruption costs minutes rather than the job.

D Cluster placement group

Optimises network latency between instances and concentrates them in one AZ, making a capacity shortfall more likely, not less.

THE SIGNAL

Replacements interrupted within minutes is the giveaway for lowest-price. It refills from the cheap pool that just emptied.

A company runs a large Java microservices estate on Linux on M6i instances and wants to reduce compute cost. The services are containerised on ECS and have no native dependencies. A separate reporting service runs on Windows, and a nightly training job uses GPU instances.

Finance wants the largest saving with the least risk.

What should the solutions architect recommend?

- ☐ A Migrate everything to Graviton instances
- ☐ B Migrate the Java services to Graviton; leave the Windows and GPU workloads on x86
- ☐ C Keep everything on x86 and buy a Compute Savings Plan instead
- ☐ D Move the Java services to T-family instances to cut cost

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A** Everything to Graviton
Impossible. Graviton has no Windows support and no GPU-attached types, so two of the three workloads cannot move.
- B** Java to Graviton, rest on x86
Java on Linux with no native dependencies is the lowest-risk Graviton migration there is. The other two stay where they must.
- C** Savings Plan instead
Worth doing as well, and it is a pricing change rather than an efficiency one. The two combine; they are not alternatives.
- D** T-family
Burstable instances under sustained microservice load exhaust credits and throttle. Cheaper per hour and slower when it matters.

THE SIGNAL

The Windows and GPU workloads are planted to test whether you know Graviton's two hard limits.

A computational fluid dynamics workload runs across 40 instances that exchange data continuously using MPI. Performance is far below expectations and profiling shows the bottleneck is inter-node network latency.

The instances are currently spread across three Availability Zones for resilience.

What should the solutions architect recommend?

- ☐ A A spread placement group across the three Availability Zones
- ☐ B A cluster placement group in a single Availability Zone, with Elastic Fabric Adapter
- ☐ C A partition placement group
- ☐ D Larger instances in the same three Availability Zones

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A

Spread placement group

Deliberately separates instances onto distinct hardware, which is the opposite of what a latency-bound workload needs.
- B

Cluster group plus EFA

Packs the nodes close together in one AZ and gives MPI a low-latency bypass of the normal network stack.
- C

Partition group

Separates instances into racks for distributed data stores. It reduces correlated failure rather than latency.
- D

Larger instances

More CPU and memory per node, and the bottleneck is the network between nodes. It may worsen the ratio.

THE SIGNAL

Spreading across AZs is the cause here, not the solution. Tightly coupled HPC trades availability for latency, deliberately.

When the Auto Scaling group scales in, in-flight requests are dropped and the terminating instance's local log files are lost before they are shipped.

The team needs up to three minutes to drain connections and flush logs before an instance goes away.

What should the solutions architect implement?

- ☐ A An EC2 termination lifecycle hook that holds the instance in Terminating:Wait while a script drains and flushes
- ☐ B Increase the load balancer's deregistration delay to 300 seconds
- ☐ C Enable instance scale-in protection on all instances
- ☐ D Suspend the Terminate process on the Auto Scaling group

◆ PAUSE AND DECIDE

What should the solutions architect implement?

- A** Termination lifecycle hook
Holds the instance in a wait state until the script completes or the timeout expires, covering both draining and log flushing.
- B** Deregistration delay
The strong near miss. It genuinely fixes the dropped requests, and does nothing for the logs, because that is a load balancer setting.
- C** Scale-in protection
Stops instances being chosen for termination at all, so the group can never scale in. It prevents the event rather than handling it.
- D** Suspend Terminate
Breaks scale-in entirely, and also breaks replacement of unhealthy instances, which is worse.

THE SIGNAL

Two problems are described and only one option addresses both. Deregistration delay is half an answer.

What to carry out of this module

- ◆ **Describe the fleet, do not list it.** Attribute-based selection plus mixed instances is the professional-level answer.
- ◆ **Separate the trigger from the delay.** A slow boot needs a warm pool; a late trigger needs a different policy.
- ◆ **Predictive runs alongside target tracking**, not instead of it. Scheduled actions cannot vary by day.
- ◆ **price-capacity-optimized, and diversify.** Graviton is Linux and arm64 only: no Windows, no GPU.

ON THE EXAM

Cluster placement groups trade availability for latency deliberately. That is the one place where spreading across AZs is the wrong instinct.